

プログラミング⑤

(アルゴリズム：数値計算と近似)

2026 年度高 2 情報

2026/06/13

微分をコンピュータに計算させる（数値微分）

微分の定義のおさらい

関数の特定の点における「傾き（接線）」を求めるのが微分である．

$$f'(x) = \lim_{h \rightarrow 0} \frac{f(x+h) - f(x)}{h}$$

コンピュータは極限を直接計算できないため、**非常に小さな数** h を代入して「近似的な傾き」を求めさせる．これを**数値微分**と呼ぶ．

数値微分の実装例

$f(x) = x^2 - 2$ の $x = 3$ における傾きを計算してみる.

```
public class NumericalDiff {  
    public static double F(double x) { return x * x - 2.0; }  
  
    // 数値微分 f'(x)  
    public static double Df(double x) {  
        double h = 1e-5; // 0.00001  
        return (F(x + h) - F(x)) / h;  
    }  
  
    public static void Main() {  
        // x=3 のときの傾きは本来 2*x = 6  
        System.Console.WriteLine("傾きの近似値: " + Df(3.0));  
    }  
}
```

ニュートン法の数学的な説明

接線の方程式から漸化式を導く

点 $(x_n, f(x_n))$ における接線の方程式は次のように表せる.

$$y - f(x_n) = f'(x_n)(x - x_n)$$

ニュートン法では、接線が x 軸と交わる点 ($y = 0$) を次の近似値 x_{n+1} とする.

$$-f(x_n) = f'(x_n)(x_{n+1} - x_n) \implies x_{n+1} = x_n - \frac{f(x_n)}{f'(x_n)}$$

ニュートン法の実装例

$\sqrt{2}$ (つまり $x^2 - 2 = 0$ の解) を高速に計算する.

```
public class NewtonMethod {  
    public static double F(double x) { return x * x - 2.0; }  
    public static double Df(double x) { return (F(x + 1e-5) - F(x)) / 1e-5; }  
  
    public static void Main() {  
        double x = 3.0; // 初期値  
        double epsilon = 1e-10; // 許容誤差  
        while (System.Math.Abs(F(x)) > epsilon) {  
            x = x - F(x) / Df(x); // ニュートン法の更新式  
            System.Console.WriteLine("近似値: " + x);  
        }  
        System.Console.WriteLine("最終結果: " + x);  
    }  
}
```

二分法

イメージ

正解が範囲 $[a, b]$ の間にあるとき, 常に中点 c を調べ, 範囲を半分ずつにしていくのが**二分法**である.

- **長所**: 関数が連続であれば, 失敗せず**必ず**解にたどり着く.
- **短所**: 一歩ずつしか近づかないため, ニュートン法に比べると遅い.

特徴	二分法	ニュートン法
速度	普通 ($O(\log N)$)	猛烈に速い
確実性	ほぼ 100%見つかる	初期値次第で失敗する

二分法の実装例

$\sqrt{2}$ (つまり $x^2 - 2 = 0$ の解) を二分法で計算する.

```
public class BisectionMethod {
    public static double F(double x) { return x * x - 2.0; }

    public static void Main() {
        double a = 1.0; // 探索範囲の左端
        double b = 2.0; // 探索範囲の右端
        double epsilon = 1e-10; // 許容誤差
        while ((b - a) > epsilon) {
            double c = (a + b) / 2.0; // 範囲の中点を計算
            // 左端と中点で関数の符号が違えば、解は左半分にある
            if (F(a) * F(c) < 0) {
                b = c; // 右端を中点まで狭める
            } else {
                a = c; // 左端を中点まで狭める
            }
            System.Console.WriteLine("近似値: " + c);
        }
        System.Console.WriteLine("最終結果: " + (a + b) / 2.0);
    }
}
```

勾配法

方程式の解ではなく最小値を見つける

ニュートン法などは「 $f(x) = 0$ となる場所」を探した. しかし, AIの世界では「エラー（誤差）が一番小さくなる谷底」を探すことが重要である.

アイデア：傾きと逆方向に進む

- 傾き $f'(x)$ が**プラス**なら, 谷底は**左**にあるので少し戻る.
- 傾き $f'(x)$ が**マイナス**なら, 谷底は**右**にあるので少し進む.

漸化式： $x_{n+1} = x_n - \alpha f'(x_n)$ (α は**学習率**)

勾配法（勾配降下法）の実装例

$f(x) = x^2 - 4x + 5$ の最小値（谷底は $x = 2$ ）を探す.

```
public class GradientDescent {  
    public static double F(double x) { return x * x - 4.0 * x + 5.0; }  
    public static double Df(double x) { return (F(x + 1e-5) - F(x)) / 1e-5; }  
  
    public static void Main() {  
        double x = 0.0; // 初期値  
        double alpha = 0.1; // 学習率（一步の幅）  
        for (int i = 1; i <= 30; i++) {  
            x = x - alpha * Df(x); // 傾きと逆方向へ進む  
            System.Console.WriteLine($"{i}回目: x = {x}");  
        }  
        System.Console.WriteLine("最終結果: " + x);  
    }  
}
```

AI と勾配法の関係

なぜこれが「AI」に繋がるのか？

最新の AI（ChatGPT 等）も、根本的にはこれと同じ原理を用いている.

1. AI が予測を行い, 正解との「ズレ (誤差)」を計算する.
2. その誤差を「関数」と見なし, 微分を使って誤差が小さくなる方向へパラメータを動かす.
3. これを何兆回と繰り返すことで, 正解率を高めていく.

このとき, 歩幅である「学習率 α 」が適切でないと, AI は賢くならない.

問題1：ニュートン法の活用と弱点の観察

問題1

先ほどのニュートン法のプログラムの関数 $F(x)$ を書き換え、以下の実験を行え.

- (1) $f(x) = x * x * x - 2.0 * x + 2.0 = x^3 - 2x + 2$ とし, 初期値 $x_0 = 5.0$ で実行して解を一つ求めよ.
- (2) $f(x) = x * x * x - 2.0 * x + 2.0 = x^3 - 2x + 2$ とし, 初期値 $x_0 = 2.0$ で実行して解を一つ求めよ. また, 解が収束しない理由について調べてみよ.
- (3) $f(x) = x - \cos(x)$ となる方程式の解を求めよ. 初期値は各自で工夫すること. (※ C#では `System.Math.Cos(x)` を用いる)

問題2：AIの学習パラメータの調整

問題2

先ほどの「勾配法」のプログラムを用いて、以下の実験を行え．

1. 学習率 α を 0.01 などの小さな値にすると、30 回後の結果はどうなるか．
2. 学習率 α を 1.1 などの大きな値に設定すると、 x の値はどう変化するか観察せよ．

この実験から、「適切な学習率」がなぜ重要であることを考察せよ．

まとめ

- **数値微分**により, コンピュータは「傾き」を自ら計算できる.
- **ニュートン法**と**二分法**には, 速度と確実性のトレードオフがある.
- **勾配法**は, 関数の谷底を探すアルゴリズムであり, 現代の AI の学習を支える最も重要な技術のひとつである.